Sprint 3 – Sprite Batch

Student Information

Integrity Policy: All university integrity and class syllabus policies have been followed. I have neither given, nor received, nor have I tolerated others' use of unauthorized aid.

I understand and followed these policies: ◯ Yes ◯ No

Name:

Date:

Submission Details

Final ***Changelist*** number:

Verified build: ◯ Yes ◯ No

Required Configurations: DEBUG only

YouTubeLink:

Discussion (What did you learn):

Verify Builds

- Follow the Piazza procedure on submission
  - Verify your submission compiles and works at the changelist number.
- Verify that only MINIMUM files are submitted
  - No – Generated files
    - *.pdb, *.suo, *.sdf, *.user, *.obj, *.exe, *.log, *.pdb, *.db, *.user
    - Anything that is generated by the compiler should not be included
  - No – Generated directories
    - /Debug, /Release, /Log, /ipch, /.vs
- Typical files project files that are required
  - *.sln, *.csproj, *.cs,
  - App.config, AssemblyInfo.cs, CleanMe.bat
  - Resources Directory:
    - *.tga, *.dll, *.wav, *.glsl, *.azul

Standard Rules

Submit multiple times to Perforce

- Submit your work as you go to perforce several times
  - Design Patterns – no minimum
  - Sprints – minimum of 5 real submissions (generally 10+)
    - As soon as you get something working, submit to perforce
    - Have reasonable check-in comments
      - Points will be deducted if minimum is not reached

Submission Report

- Fill out the submission Report
  - No report, no grade

Code and project needs to compile and run

- Make sure that your program compiles and runs
  - Warning level 4
  - NO Warnings or ERRORS
    - Your code should be squeaky clean.
  - Code needs to work “as-is”.
    - No modifications to files or deleting files necessary to compile or run.
  - All your code must compile from perforce with no modifications.
    - Otherwise it's a 0, no exceptions

Project needs to run to completion

- If it crashes for any reason…
  - It will not be graded and you get a 0

No Containers

- Containers (No automatic containers or arrays
- Template or generic parameters
- No arrays
  - You need to do this the old fashion way - ***YOU EARNED IT***

Leave Project Settings

- Do NOT change the project or warning level
  - Any changing of level or suppression of warnings is an integrity issue

Simple C#

- No .Net
- We are using the basics
  - Types:
    - Class, Structs, intrinsic types (int, float, bool, etc…)
    - NO arrays allowed!
  - Basics language features
    - Inheritance, methods, abstract, virtual, etc…

No Debug code or files disabled

- Make sure the program has only active code
  - If you added debug code or commented out code,
    - please return to code to active state or remove it

Adding files to this project

- Make sure you add the files in the appropriate sub-directories
- Make sure any new files are successfully integrated into the project
- Make sure your new files are submitted to Perforce

Due Dates

- See Piazza for due date and time
- Submit program perforce in your student directory assignment supplied.
- Fill out your this ***Submission Report*** and commit to perforce
  - ***ONLY*** use Adobe Reader to fill out form, all others will be rejected.
  - Fill out the form and discussion for full credit.

Goals

- SpriteBatch – development
  - Adding Heterogeneous types in one manager
  - Mixture of both BoxSprites and GameSprite
- Add Priority to Sprite Batch
  - Each batch has a priority that controls the ordering on the list
- Refactor ***Sprite*** to ***GameSprite***
  - Change the file and class names
    - ***Sprite*** change to ***SpriteGame***
    - ***SpriteMan*** change to ***SpriteGameMan***

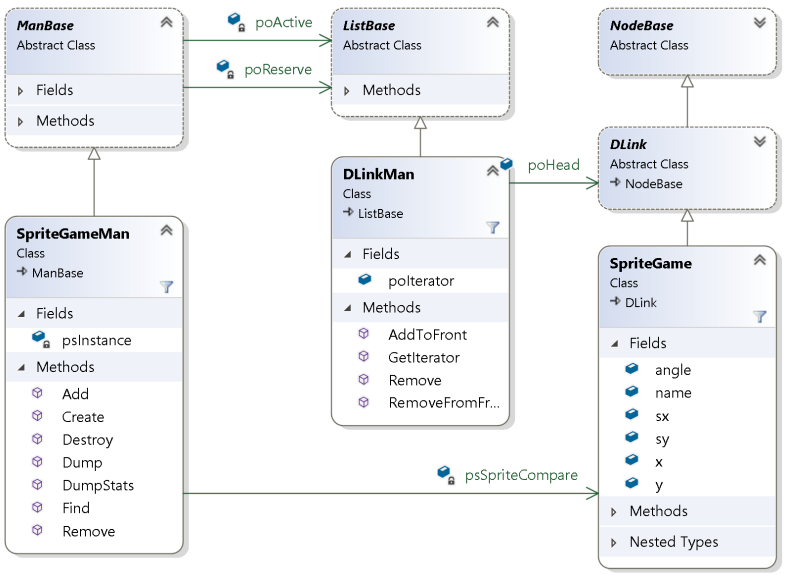
Assignments

Grading

- This sprint you are going to learn/modify the code from class.
- There are 3 steps to this refactoring
- Start with your existing sprint and extend, add, modify for this sprint

Step 1 – Refactoring Names

- Change the name of the class
- Change the name of the file
  - ***Sprite*** change to ***SpriteGame***
  - ***SpriteMan*** change to ***SpriteGameMan***
- ***Demostrate by showing the Solution View and SpriteGameMan UML diagram in tool***



Step 2 – Add Priority number to SpriteBatch

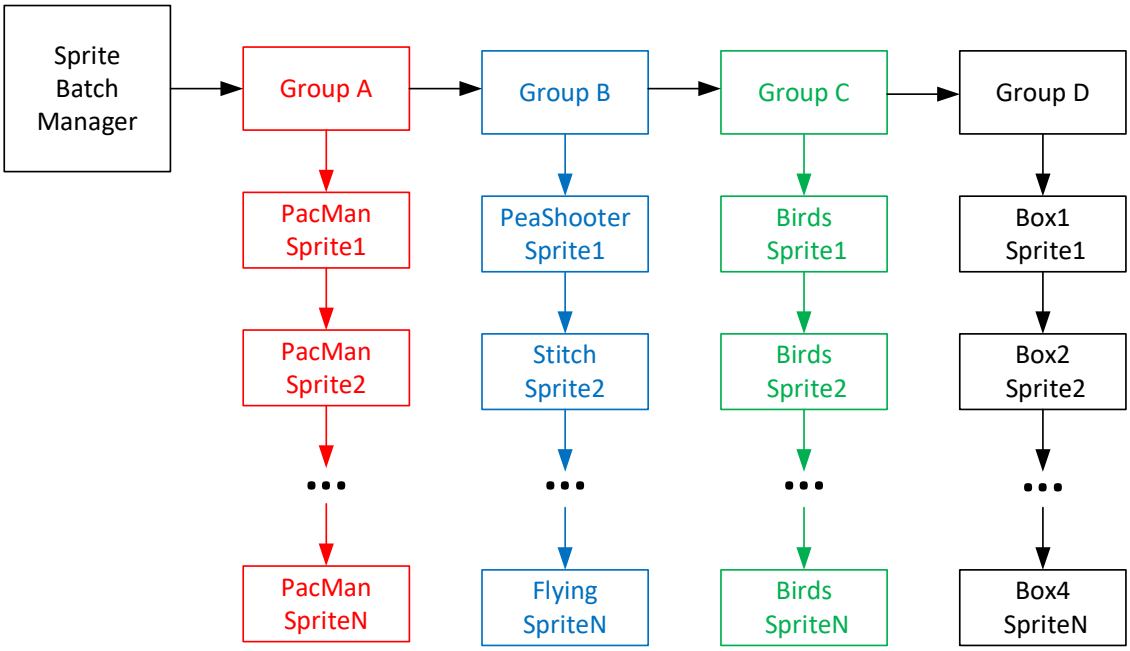
- Change the order of the groups dynamically
  - Insert the groups with an "priority" number
    - Just an index you can use for the Group insert sort.
  - Create a convention… example: small "priority" groups drawn before large numbers priorities
    - GroupA (priority:5),
    - GroupB (priority:50),
    - GroupC (priority:300),
    - GroupD (priority:600)
      - Draw ordering is… GroupA, then GroupB, then GroupC then GroupD
- If you want to change an ordering…
  - Say you want Group A to be drawn after Group C.
  - Change Group A priority to 500 and move the group in the linked list
  - Now it would be GroupB, GroupC, GroupA
    - Draw ordering – B, C, A, D
- ***Demostrate the ordering change the by simply changing the priority number***
  - Show the game changing order (like in class)

Step 3 – Heterogeneous SpriteBatch

- Create SpriteBox Manager
  - Similar to SpriteGameMan – but manages Sprite Boxes
- Have your sprite batch manager handle heterogeneous sprites
  - Extend your Sprite Batch Manager to handle at least two different types of sprites in one manager.
    - SpriteGame – the ones you know {AngryBird Yellow example}
    - SpriteBox – Wireframe box constant color White, Yellow or some color
- Sprites Batch Manager
  - Create 4 groups of 4 sprites
    - More sprites or more groups are allowed
  - Create the Sprite Batch Manager
    - It associates sprites by groups (batch)
      - Attach sprites one by one to a specific group
    - Order within on group is not needed
      - Push to front is fastest
    - Control the ordering by changing the Priority of each SpriteBatch (group)
- ***Make sure you demo… you see overlapping***
  - Then change the priority and see the overlapping change

Sprite Batch

- Group A – PacMan
- Group C – Angry Birds
- Group B – Misc (Stitch, PeaShooter, Skeleton, Flying)
- Group D – Boxes (4 boxes different colors)



Required Features

Features to show off in Demo

Show it off and talk about these required features

- Show off the 3 steps
  - Demo and talk in the video
- Show it off
  - Be proud of what you did
- Show the UML diagram of SpriteBatch()
  - Talk about the priority system

Video Capture

- Have Visual Studio open and ready to go.
- Start recording
  - Show output window running

  - Speak clearly and loud
  - Discuss the features you added in this demo
  - Duration
    - Approx. 2-3 min long YouTube video capture
      - Do not exceed 5 min
    - Use Zoom recording
      - High resolution
      - Good sound quality
      - Show on the capture… it compiling and then running
    - Place YouTube link in PDF
    - Make sure it public and runs without passwords or login
    - Watch your video
      - Can you see the code clearly
      - Can you hear your voice clearly
- Don't lose points
  - Make sure everything is submitted properly
  - Don't record beyond 5 min.
  - You audio volume was too quiet (test it and verify it)
  - Your YouTube link needs to work "As-IS" with no login
    - Don't make it private… make it unlisted
  - You check-in too many files or temporary files
    - Make sure you run CleanMe.bat before perforce submission

General guidelines:

- Post on Piazza questions and clarifications
- Make sure you delete these using directives (we are not using them)
  - `using System.Collections.Generic;`
  - `using System.Linq;`
  - `using System.Text;`
  - `using System.Threading.Tasks;`

Development

- Copy your Sprint2 into Sprint3.
  - Submit to perforce and start your work
- Do not copy temporary files or submit extra data
- Make sure your run CleanMe.bat before adding to perforce

Submission

- Submit your SprintX directory into perforce:
  - /student/<yourname>/SprintX/...
    - You need to submit a complete C# project
  - Solution, project and C# files (whatever it takes to build the project)

        - Do not submit anything that is auto generated
    - Run the supplied CleanMe.bat before submission
        - Should cleanup files
- Fill out the Submission report and submit that pdf to your student directory

Validation

Simple checklist to make sure that everything is submitted correctly

- Is the project compiling and running without any errors or warnings?
- Does the project runs ***ALL*** without crashing?
- Is the submission report filled in and submitted to perforce?
- Follow the verification process for perforce
    - Is all the code there and compiles "as-is"?
    - No extra files
- Submit the YouTube link to PDF

Hints

Most assignments will have hints in a section like this.

- Watch the lecture… look at the code supplied in class
- Study the projects in class lecture

Troubleshooting

- Print to the output window to help you debug
    - It really helps
- Ask for clarification on piazza